Lecture Slides: Computer organization and assembly language

• Computer Organization & Assembly Language - Lecture Slides**

• Target audience: Tertiary-level Computer Engineering / Computer Science students*

•

Slide 1 - Title & Scope

• Computer Organization and Assembly Language**

• **Focus:** How the physical structure of a modern processor shapes the ARMv7 instruction set and how assembly language exposes that structure.

• **Key Themes**

1. Relationship between hardware organization & instruction execution.

2. Core components of the ARMv7 ISA (register file, pipeline, exception model).

3. Practical impact of low-level programming on performance, power, and security.

•

Slide 2 - Learning Objectives

By the end of the session you will be able to:

1. **Describe** the functional blocks of a CPU and **explain** the fetch-decode-execute-write-back (FDEWB) cycle as realized in hardware.

2. **Identify** the purpose of the ARMv7 registers (PC, LR, SP, R0-R12) and the calling convention.

3. **Write** a correct ARM sub-routine using `BL`, `BX LR`, `PUSH`, and `POP` that obeys the ARM-EABI rules.

4. **Analyse** the bus system, memory hierarchy, and I/O controllers and their effect on data movement and latency.

5. **Evaluate** trade-offs (register vs. stack, conditional branch vs. predication, power vs. performance).

•

Slide 3 - Why Hardware-Software Co-Design Matters

• **Computer organization** = concrete arrangement of CPU, caches, buses, and I/O.

• **ISA** = abstract contract; implementation details (pipeline depth, cache size, bus width) dictate timing and resource usage.

• **Embedded & systems programmers** must predict these effects to write deterministic, efficient code.

• **ARMv7** dominates mobile/embedded markets -> an ideal vehicle for illustration.

•

Slide 4 - CPU Architecture & the Complete Execution Cycle

• Core blocks** - ALU, register file, PC, control unit, pipeline registers, bus interfaces.

| Stage | What happens | Hardware support |

|------|--------------|------------------|

| **Fetch** | PC places address on the address bus -> instruction word read into the Instruction Register (IR). | Instruction cache (I-cache) + address generation unit |

| **Decode** | Control unit reads opcode in IR, selects functional units, determines operand sources (register, immediate, memory). | Decoder, register-file read ports |

| **Execute** | ALU performs arithmetic/logic, memory interface conducts load/store, or branch logic computes a new PC. | ALU, load/store unit, branch unit |

| **Memory** *(optional for load/store)* | Data cache (D-cache) supplies or receives the operand/value. | D-cache, bus bridge |

| **Write-Back** | Result written to destination register or memory; PC updated for next fetch. | Register-file write ports, store buffer |

| **Pipeline view** | 5-stage pipeline (IF-ID-EX-MEM-WB) overlaps the stages so that >1 instruction retires per clock. | Pipeline registers, hazard detection unit |

•

Slide 5 - Memory Hierarchy & Cache Subsystem

• **Levels**: Registers -> L1 I-cache / L1 D-cache -> L2 (unified) -> Main RAM -> Secondary storage.

• **Goal:** Reduce average memory latency by exploiting **temporal** (reuse) and **spatial** (nearby) locality.

• **Cache policies**

• Write-back vs. write-through.

• Replacement: true LRU, pseudo-LRU, random.

• **Line size** - typically 32 B or 64 B; determines granularity of transfers between L1/L2 and RAM.

• **Assembly impact**

• Align data structures to cache-line boundaries (`.align 6`).

• Avoid false sharing by separating frequently updated variables.

• Use `PLD` / `PREFETCH` hints for critical loops.

•

Slide 6 - System Bus & I/O Interconnects

• **Bus families** (AMBA AHB/APB, AXI, PCIe) provide separate **address**, **data**, and **control** channels.

• **Master-slave arbitration** - CPU, DMA engines, and peripherals compete; arbiter grants one master per cycle (round-robin, priority).

• **Memory-mapped I/O** - peripherals appear as addresses; a read/write triggers device actions.

• **Programmed I/O vs. DMA** -

• *PIO*: CPU performs every data transfer -> high latency, high CPU load.

• *DMA*: Dedicated engine moves blocks of data -> lower latency, frees CPU.

• **Illustration** - Block diagram: CPU ↔ AHB bus ↔ L2 cache ↔ DDR, plus APB peripheral block (UART, GPIO).

•

Slide 7 - ARMv7 Register File & Calling Convention (AAPCS/EABI)

• General-purpose registers**

• R0-R12 - caller-saved, used for arguments & temporaries.

• R13 = **SP** (stack pointer).

• R14 = **LR** (link register - holds return address).

• R15 = **PC** (program counter).

• Special registers**

• CPSR - condition flags, mode bits, interrupt masks.

• SPSR - saved CPSR for exception handling.

• Calling convention**

| Role | Registers |

|---|---|

| Argument passing (first 4 integer args) | R0-R3 |

| Return value | R0 |

| Callee-saved (preserved across calls) | R4-R11, SP |

| Caller-saved (may be clobbered) | R0-R3, R12, LR |

| Stack growth | Downwards (SP decremented) |

| Stack alignment | 8-byte (ARM) or 16-byte (AArch64) boundary |

•

Slide 8 - Building a Correct ARM Sub-routine (EABI-compliant)

```asm

.text

.global add_two

add_two: @ <--- entry point

PUSH {R4, LR} @ save callee-saved R4 and return address

MOV R4, R0 @ preserve first operand (R0 is caller-saved)

ADD R0, R1, R4 @ R0 = R1 + saved operand -&gt; return value

POP {R4, PC} @ restore R4, return (PC←LR)

```

• **Prologue** - creates stack frame, saves LR (and any callee-saved regs).

• **Body** - operates only on registers -> no extra memory traffic.

• **Epilogue** - `POP {..., PC}` restores LR into PC, equivalent to `BX LR`.

•

Slide 9 - Branching, Predication & Pipeline Effects

• **Unconditional/Conditional branches** - `B`, `BEQ`, `BNE`, ... use condition-code flags (N, Z, C, V).

• **Branch with Link (`BL`)** - PC-relative, stores return address (PC+4) in LR; enables position-independent code.

• **Branch prediction** - static (predict not-taken) vs. dynamic (2-bit saturating counter).

• **Branch penalty** - pipeline flush if target unknown early; typical 2-3 cycle cost in a 5-stage pipeline.

• **Predicated execution (`IT` block)** - up to four instructions executed conditionally without a branch, reducing misprediction cost.

• **Example IT block**

```asm

IT EQ @ Execute next instruction only if Z flag set

ADDEQ R0, R0, #1 @ Increment R0 conditionally

```

•

Slide 10 - Trade-off Analysis (Performance, Power, Complexity)

| Trade-off | What it means | Typical impact on ARM code |

|-----------|---------------|----------------------------|

| **Register vs. Stack** | Registers are fast, limited; stack provides virtually unlimited storage. | Keep hot variables in R0-R12; spill to stack only when necessary (e.g., >4 arguments). |

| **Conditional branch vs. Predication** | Branches may cause pipeline flush; predication avoids flush but can waste execution slots. | Use `IT` for short, highly predictable conditions; prefer branches for long or unpredictable paths. |

| **Power vs. Performance** | Deeper pipelines & higher clock rates increase power; aggressive caching improves performance but adds leakage. | Write tight loops that allow the CPU to stay in low-power states (e.g., `WFI`); avoid unnecessary memory traffic. |

| **Static vs. Dynamic Scheduling** | Static (compiler-controlled) is simpler; dynamic (out-of-order) yields higher IPC but higher silicon cost. | ARMv7 Cortex-A series is in-order -> rely on instruction ordering and manual unrolling for ILP. |

•